

T_1 再次按相同条件读取数据时,发现某些记录“神秘地”消失了。

② 事务 T_1 按一定条件从数据库中读取某些数据记录后,事务 T_2 插入了一些记录,当 T_1 再次按相同条件读取数据时,发现多了一些记录。

本章重点讨论前三类数据不一致性。读者如果对幻读有兴趣,可以参考相关文献。

产生上述数据不一致性的主要原因是并发操作破坏了事务的隔离性。并发控制机制就是要用正确的方式调度并发操作,使一个用户事务的执行不受其他事务的干扰,从而避免造成数据的不一致性。

另一方面,对数据库的应用有时允许某些不一致性,例如有些统计工作涉及的数据量很大,读到一些“脏”数据对统计精度没什么影响,这时可以降低对一致性的要求以减少系统开销。

并发控制的主要技术有封锁(locking)、时间戳(timestamp)、乐观方法(optimistic scheduler)和多版本并发控制(multi-version concurrency control, MVCC)等。本章讲解基本的封锁技术,它也是众多数据库产品采用的基本方法。

12.2 事务的隔离级别

为防止数据出现不一致性,需要数据库管理系统对并发操作进行控制。这种控制越严格,事务的隔离性就越强,数据的一致性就越有保障,但系统的效率也会随之下降。在实际应用中,不同应用场景对数据一致性的要求并不相同,所以在 SQL 标准中给出了事务的 4 类隔离级别,以满足不同应用场景的需求。这 4 类隔离级别由低到高分别是:读未提交、读已提交、可重复读和可串行化。它们都能有效避免丢失修改,但对其他数据一致性的保障程度各异。

“读未提交”是允许一个事务可以读取另一个未提交事务正在修改的数据。它可能出现脏读、不可重复读和幻读的情形。

“读已提交”是只允许一个事务读其他事务已提交的数据。显然,它可以有效避免脏读,但是它不能保证可重复读和不幻读。

“可重复读”是一个事务开始读取数据后,其他事务就不能再对该数据执行 UPDATE 操作了。由于脏读和不可重复读是因一个事务读取数据,而另一个事务对该数据进行修改操作造成的,“可重复读”杜绝了这种情形的产生。但此时其他事务仍然可以执行 INSERT 和 DELETE 操作,所以它不能保证不幻读。

“可串行化”是最高的事务隔离级别。在该级别下,事务执行顺序是可串行化的,可以避免丢失修改、脏读、不可重复读和幻读。事务的可串行化将在 12.6 节中详细介绍。

事务的 4 个隔离级别与数据不一致性的关系见表 12.1 所示。

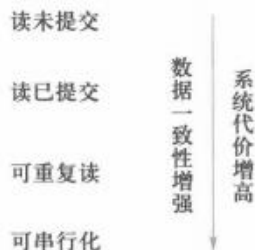
例如,若数据库管理系统的事务隔离级别是“读未提交”,则不可能产生丢失修改,但可能产生脏读、不可重复读和幻读的情况。若为“读已提交”,则不会产生丢失修改、脏读,但可能产生不可重复读和幻读的情况。目前大多数数据库默认的事务隔离级别是“读已提交”,如 Kingbase、SQL Server、Oracle 等;而 MySQL 的默认级别是“可重复读”。

表 12.1 事务隔离级别与数据不一致性的关系

事务隔离级别	数据不一致性			
	丢失修改	脏读	不可重复读	* 幻读
读未提交	否	是	是	是
读已提交	否	否	是	是
可重复读	否	否	否	是
可串行化	否	否	否	否

事务隔离级别并不是越高越好，它与数据一致性以及系统代价的关系如图 12.3 所示。数据库管理系统通常都提供了设置事务隔离级别的方法，在实际中，用户应根据应用的特点和需求选择合适的事务隔离级别。

二维码所示内容通过实验来演示 4 种事务隔离级别下丢失修改、脏读、不可重复读等数据不一致的情况，读者可以直观地感受各种隔离级别在事务中的作用。



扩展阅读：隔离级别实验

图 12.3 事务隔离级别与数据一致性及系统代价的关系

12.3 封锁

封锁是实现并发控制的一种非常重要的技术。所谓封锁，就是事务 T 在对某个数据对象操作之前，先向系统发出请求，对其加锁。加锁后事务 T 就对该数据对象有了一定的控制，在事务 T 释放它的锁之前，其他事务不能更新或读取此数据对象。例如，在例 12.1 中，事务 T_1 要修改 A ，若在读出 A 前先锁住 A ，其他事务就不能再读取和修改 A 了，直到 T_1 修改并写回 A 后解除了对 A 的封锁为止。这样，就不会丢失 T_1 的修改。

确切的控制由封锁的类型决定。基本的封锁类型有两种：排他型锁(exclusive lock，简称 X 锁)和共享型锁(shared lock，简称 S 锁)。

排他型锁又称为写锁(write lock)。若事务 T 对数据对象 A 加上 X 锁，则只允许 T 读取和修改 A ，其他任何事务都不能再对 A 加任何类型的锁，直到 T 释放 A 上的锁为止。这就保证了其他事务在 T 释放 A 上的锁之前不能再读取和修改 A 。